

Workday and AWS Integrations

Table of Contents

1. Workday and AWS Integrations	2
Workday Extend and AWS native integrations	2
Workday Extend and AWS custom integrations.....	3
Amazon Bedrock AgentCore and Workday (A2A and MCP) agent integrations.....	5
Workday Agents and AWS MCP server integrations	5
2. AWS Environment Setup	7
Why this matters.....	7
AWS Cloud9 (No local installs)	8
CloudShell (No setup)	8
Local Workstation (Windows/macOS).....	8
3. Sample Demos	9
3.1. Demo 1: Getting Started with Workday Extend and AWS Lambda	9
3.2. Demo 2: Real-Time Translation in Workday with AWS Translate and Comprehend	11
3.3. Demo 3: Amazon Bedrock AgentCore — Employee Onboarding Sample	12
3.4. Demo 4: AI Gateway to Amazon Bedrock with API Gateway	13
4. Additional Samples	15
4.1. CloudFormation Fundamentals	15
4.2. Provision an Amazon S3® Bucket.....	15
4.3. Deploy a "Hello from Lambda" Function	17
4.4. Expose Lambda via AWS API Gateway	18
5. Troubleshooting: Top Tips	22
6. Further Reading.....	24

Audience: Workday Developers - new to AWS (100) through seasoned architects (400).

Purpose: This document provides step-by-step instructions and code samples for using AWS services in Workday Extend/Build and with Workday agents and MCP tools (Workday Sana Self-Service Agent, Workday Agent System of Record, Workday Agent Gateway). You can use AWS services in your Workday applications in the following ways (Section 1).

Note: To build Extend applications using AWS native integrations (Section 1A), please use the AWS account that comes with your Workday Extend hackathon tenant. For all other AWS integrations (Sections 1B, 1C, 1D), please use the AWS accounts provided in **Section 2**.

1. Workday and AWS Integrations

AWS-Workday Samples GitHub Repo: <https://github.com/aws-samples/sample-workday-aws-integrations>

Workday Extend and AWS native integrations

This uses the AWS components natively built in to Workday Orchestrate – Amazon S3, AWS Lambda, and Amazon EventBridge. You can use additional AI services – Amazon Textract, Amazon Translate, Amazon Comprehend, and Amazon Recognition from AWS Lambda to build AI-powered Extend applications.



For more details please see:

<https://github.com/aws-samples/sample-workday-aws-integrations/tree/main/workday-extend-examples>

<https://github.com/aws-samples/sample-workday-aws-integrations/tree/main/workday-extend-examples/docs>

Additional documentation:

- AWS Starter Kit - <https://developer.workday.com/app-catalog/awsStarterKit>
- AWS Badge Generator - <https://developer.workday.com/app-catalog/AWSBadgeGenerator>
- AWS Driver's License Processing with Translate - <https://developer.workday.com/app-catalog/AWSDriversLicenseWithTranslations>
- AWS Driver's License - <https://developer.workday.com/app-catalog/AWSDriversLicense>
- Amazon S3 - <https://developer.workday.com/documentation/GUID-b2064357-e4f4-49e7-9bd3-1191c9f5bd70-enHYPHENus/AmazonS3>
- AWS Lambda - <https://developer.workday.com/documentation/GUID-a8a83c29-7a27-434d-88a7-7c24668467e2-enHYPHENus/AWSLambda>
- Amazon Event Bridge - <https://developer.workday.com/documentation/GUID-dabef458-3efd-4073-a114-2f49d34a5a39-enHYPHENus/AmazonEventBridge>
- AI APIs - <https://developer.workday.com/documentation/GUID-0a98ed58-02ef-4dbb-9877-29932c274c7e-enHYPHENus/AWSArtificialIntelligenceAIServicees>

Sample Demos:

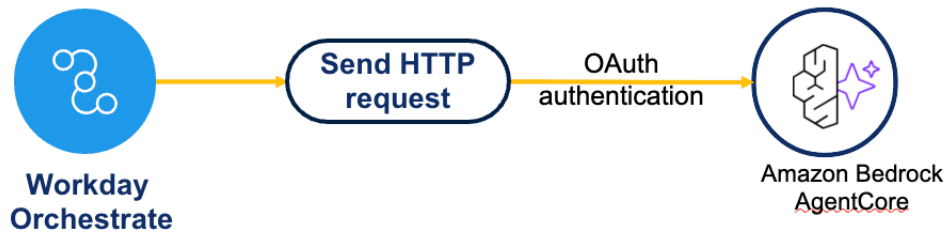
[Demo 1: Getting Started with Workday Extend and AWS Lambda](#)

[Demo 2: Real-Time Translation in Workday with AWS Translate and Comprehend](#)

Workday Extend and AWS custom integrations

A. Workday and Amazon Bedrock AgentCore API Integration

You can invoke an AI agent deployed on Amazon Bedrock AgentCore from Workday Orchestrate “Send HTTP Request” directly. In this case, you can use OAuth authentication. [Here](#) is a reference sample for bi-directional integration between Workday and AgentCore using API integration.

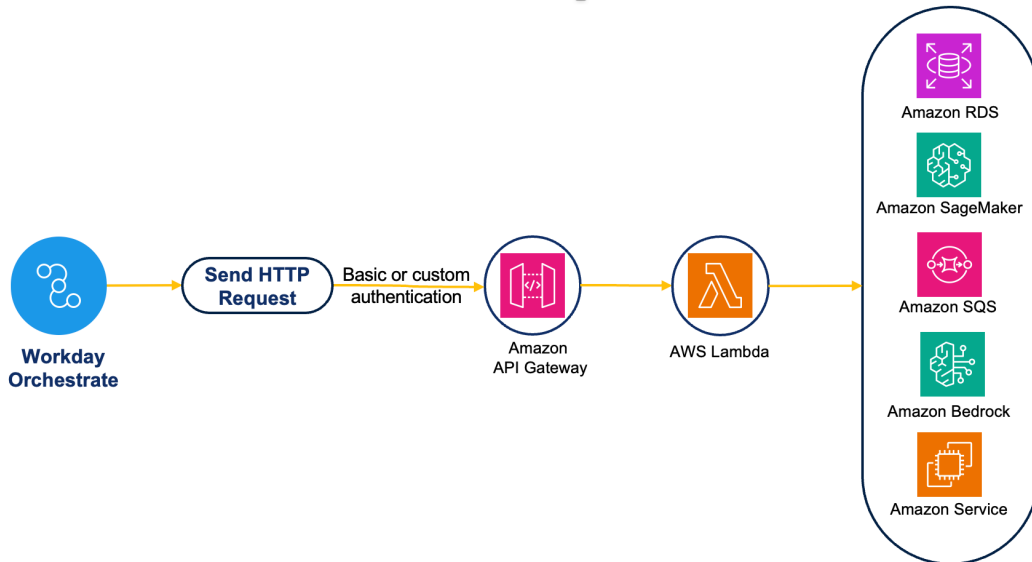


Sample Demos:

[Demo 3: Amazon Bedrock AgentCore — Employee Onboarding Sample](#)

B. Workday and AWS API Integration

Additionally, you can invoke AWS services from Workday Orchestrate “Send HTTP Request” by creating an API endpoint for that service using API Gateway and Lambda. You can use either basic or custom authentication for invoking the AWS API. [Here](#) is a reference sample for accessing a Bedrock model from Workday Extend.

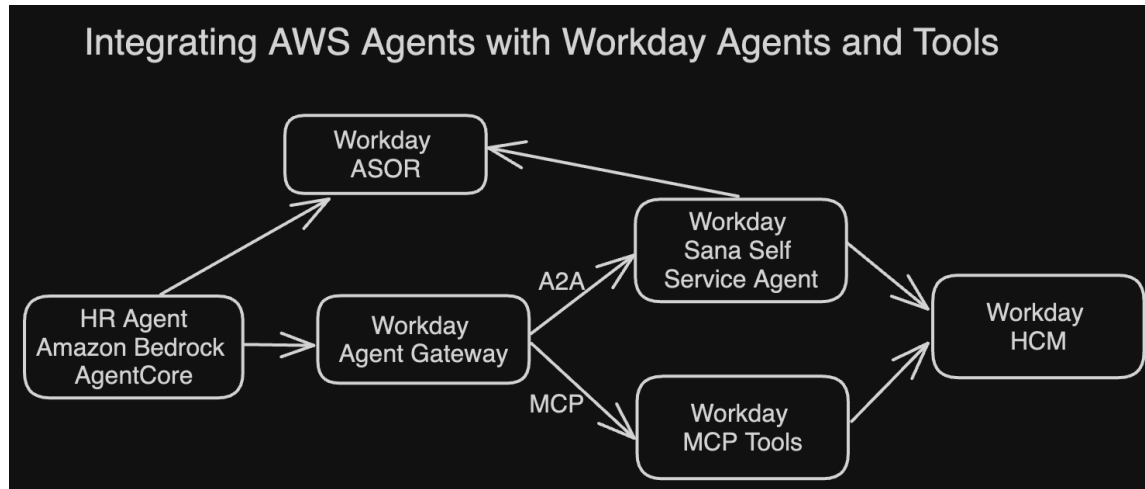


Sample Demos:

[Demo 4: AI Gateway to Amazon Bedrock with API Gateway](#)

Amazon Bedrock AgentCore and Workday (A2A and MCP) agent integrations

You can integrate your agents deployed on Amazon Bedrock AgentCore with Workday agents and MCP tools using A2A and MCP protocols. [This](#) workshop walks through the instructions. Please note that this is only a reference, and the actual implementation outside of this workshop will require additional configurations.



Workday Agents and AWS MCP server integrations

You can access AWS services over MCP using supported AWS MCP servers. Details are below.

AWS MCP Server

AWS MCP Server is a managed server that gives AI agents access to AWS through the Model Context Protocol (MCP), available via a single endpoint with CloudWatch metrics, IAM-based access controls, and CloudTrail audit logging.

What can you do with it?

Ask your AI agent to perform AWS tasks in natural language — e.g., create an S3 bucket with versioning and lifecycle policies, provision EC2 instances, deploy Lambda functions, spin up DynamoDB tables, troubleshoot CloudFormation deployments, configure CloudWatch alarms, and search AWS documentation.

Quick Installation Steps

1. Install the AWS CLI

2. Install uv if not already installed
3. Configure AWS credentials (aws login or aws configure)
4. Configure your MCP client

```
{
  "mcpServers": {
    "aws-mcp": {
      "command": "uvx",
      "timeout": 100000,
      "transport": "stdio",
      "args": [
        "mcp-proxy-for-aws@latest",
        "https://aws-mcp.us-east-1.api.aws/mcp",
        "--metadata", "AWS_REGION=us-west-2"
      ]
    }
  }
}
```

5. Test the connection by asking your agent: "What AWS Regions are available?"

For detailed instructions, see: Setting up the AWS MCP Server

(<https://docs.aws.amazon.com/agent-toolkit/latest/userguide/getting-started-aws-mcp-server.html>)

Open source MCP servers for AWS

These are a suite of specialized MCP servers that help you get the most out of AWS, wherever you use MCP.

For example, Aurora MCP Server is an open-source MCP server that lets AI agents interact with Aurora MySQL by converting natural language questions into SQL queries and executing them.

What can you do with it?

Ask your AI agent questions in natural language and it will generate and run MySQL queries against your database — e.g., "show me all orders from last month," "what are the top 10 customers by revenue," or "describe the schema of the users table."

Quick Installation Steps

1. Install uv and Python 3.10+
2. Configure AWS credentials (aws configure or environment variables)

AWS Workshop: Storage, Compute & AI Orchestration

3. Enable RDS Data API on your Aurora cluster (if using Data API method)
4. Add the MCP server entry to your client

```
{
  "mcpServers": {
    "awslabs.mysql-mcp-server": {
      "command": "uvx",
      "args": [
        "awslabs.mysql-mcp-server@latest",
        "--resource_arn", "[your data]",
        "--secret_arn", "[your data]",
        "--database", "[your data]",
        "--region", "[your data]",
        "--readonly", "True"
      ],
      "env": {
        "AWS_PROFILE": "your-aws-profile",
        "AWS_REGION": "us-east-1",
        "FASTMCP_LOG_LEVEL": "ERROR"
      },
      "disabled": false,
      "autoApprove": []
    }
  }
}
```

5. Test by asking your agent a natural language question about your database

For detailed instructions, see: Open source MCP servers for AWS
(<https://awslabs.github.io/mcp/>)

2. AWS Environment Setup

Why this matters

A consistent development environment ensures you can run AWS commands and Python scripts without extra local configuration.

Each team will have access to an individual AWS account via Workshop Studio. Each team will receive a team link to log into the workshop to gain access to an AWS account. When you initially join the workshop event, you will need to select **Email one-time password (OTP)** as your sign-in method. After entering your email address, you will receive a one-time email passcode. After entering the passcode and accepting the terms and conditions, you can join the event. Once you join the event, you can ignore the workshop instructions and access the AWS console by clicking the **Open AWS console** link shown below.

▼ AWS account access

Open AWS console
(us-east-1) 

Get AWS CLI credentials

Workshop catalog in AWS Builder
Center 

Exit event

AWS Cloud9 (No local installs)

1. Sign in → Services → Cloud9 → Create environment
2. Fill in:
 - **Name:** `cf-workshop-env`
 - **Instance type:** `t3.small` (for smoother performance)
 - **Platform:** Amazon Linux 2023
3. Create → wait ~1–2 min until the IDE opens.
 - **Note:** Cloud9 auto-manages credentials; skip `aws configure`.
 - To switch IAM roles or regions, click the cog icon → AWS Settings.

CloudShell (No setup)

1. Sign in to the Console → Service → CloudShell
2. You will be given a browser-based terminal in the AWS Console where you run commands as root with the same AWS IAM permissions as your signed-in console identity — no local setup or credential configuration needed.
3. Sessions time out after 20-30 minutes of inactivity and any files outside `~/` (your 1 GB persistent home directory) are lost when the session ends.
4. Great for quickly running `aws cli` commands in seconds with no setup required

Local Workstation (Windows/macOS)

1. **Install AWS CLI v2** (if you don't have it installed)
 - **macOS:** `brew install awscli`
 - **Windows:** Download MSI from <https://aws.amazon.com/cli/>
2. **Configure your AWS credentials**

Use the CLI export variable commands you will get from the Workshop link . On the main workshop page, click **Get AWS CLI credentials**. You should end up pasting something like the following into your CLI (may look different depending on Mac or Windows):

```
export AWS_DEFAULT_REGION="us-west-2"
export AWS_ACCESS_KEY_ID="ASIA4Z...S4X6AXLR"
export AWS_SECRET_ACCESS_KEY="242NcnMNjee...VoXV9XGVV2ZktDFq"
export AWS_SESSION_TOKEN="IQoJb3JpZ2luX2VjEMz...skjf9j23fKDS0IJF"
```

3. **Install Python & boto3** (if you don't have it installed)

- *# macOS*
brew **install** python3
 - *# Windows: run installer from python.org*
pip3 **install** boto3 pyyaml
-

3. Sample Demos

3.1. Demo 1: Getting Started with Workday Extend and AWS Lambda

Blog: https://builder.aws.com/post/3E2Ubq5C4eHW9605orImn3HbDfp_p/getting-started-with-workday-extend-and-aws-lambda

GitHub Repo: <https://github.com/aws-samples/sample-workday-aws-integrations/blob/main/workday-extend-examples/docs/blog-01-getting-started-with-lambda.md>

This demo walks through a complete, working example: a simple Lambda function invoked by a Workday Orchestration that performs addition of two numbers—the math is trivial by design, but the integration pattern of Workday Orchestration calling an AWS Lambda function through a secure external endpoint is the reusable foundation for every Workday-to-AWS integration you will build.

What it demonstrates:

- A complete Workday Extend + AWS Lambda integration end-to-end
- A Workday Orchestration invoking a Lambda function via an External Endpoint
- The reusable pattern: Workday Orchestration → External Endpoint → AWS Lambda → JSON Response → Workday

- Lambda running inside a Workday-managed VPC with IAM controls and OAuth 2.0 authentication
- Infrastructure-as-code deployment using AWS SAM
- A trivial addition function as a stand-in — swap it for real use cases like address validation, document OCR, translation, fraud detection, or AI assistance

Deployment Steps:

1. **Clone the repo** — git clone the sample-workday-aws-integrations repo and navigate to the SAM version of the Lambda Sum sample.
2. **Review the Lambda function** — A Python handler that accepts {"a": N, "b": N}, validates inputs, sums them, and returns JSON with a statusCode and body.
3. **Review the SAM template** — Defines the Lambda function with IAM role, VPC config (security group + subnets), all resolved dynamically from SSM Parameter Store (/workday-extend-tenant-config/...).
4. **Configure samconfig.toml** — Set stack_name (must use app- prefix), region, and s3_bucket matching your Workday tenant.
5. **Deploy** — sam build && sam deploy.
6. **Configure Workday Extend** — Create an Extend App, define the External Endpoint pointing to your Lambda, and build an Orchestration that sends two numbers and parses the result.

Testing the Deployment:

Local (pre-deploy):

```
echo '{"a": 10, "b": 20}' > test-payload.json
sam local invoke SumLambdaFunction --event test-payload.json
```

Expected output: {"statusCode": 200, "body": "{\"a\": 10, \"b\": 20, \"result\": 30}"}

AWS (post-deploy):

```
aws lambda invoke \
  --function-name SumLambdaFunction \
  --payload file://test-payload.json \
  --region us-west-2 \
  output.json
cat output.json
```

End-to-end: Execute the Orchestration from within Workday and verify the response flows back correctly. The repo's SUM_DEPLOYMENT_GUIDE.md has step-by-step screenshots for the Workday-side configuration.

3.2. Demo 2: Real-Time Translation in Workday with AWS Translate and Comprehend

Blog: https://builder.aws.com/post/3E2VOB47llz1m5EYrkaBh1CxrO_p/real-time-translation-in-workday-with-aws-translate-and-comprehend

GitHub Repo: <https://github.com/aws-samples/sample-workday-aws-integrations/blob/main/workday-extend-examples/docs/blog-02-real-time-translation.md>

This demo builds on Demo 3 the foundational Lambda pattern to deliver real-time, multi-language translation directly inside Workday workflows — a Lambda function receives text and a target language from a Workday Orchestration, auto-detects the source language via Amazon Comprehend, translates it using Amazon Translate across 75+ supported languages, and returns the result back into Workday, eliminating the need for manual translation services, external copy-paste, or duplicate content maintenance.

What it demonstrates:

- Real-time neural machine translation inside Workday workflows using AWS Translate (75+ languages, sub-second response)
- Automatic source language detection via Amazon Comprehend — users don't need to specify what language they're writing in
- Building on the Demo 3 Lambda pattern with the addition of AWS AI service calls and scoped IAM permissions
- Handling optional parameters (source language defaults to 'auto') and flexible input formats
- Production-ready error handling with consistent JSON responses for orchestration routing

Deployment Steps:

1. **Clone and navigate** — `cd sample-workday-aws-integrations/workday-extend-examples/workday-translate-sample/sam-version`
2. **Review the Lambda function (src/index.py)** — Accepts text, optional sourceLanguage (defaults to 'auto'), and targetLanguage; calls `translate.translate_text()` via Boto3; returns translated text with detected source language.
3. **Review the SAM template** — Same VPC/role structure as Part 1, plus a Policies block granting `translate:TranslateText` and `comprehend:DetectDominantLanguage`.
4. **Deploy** — `sam build && sam deploy`
5. **Configure Workday** — Create an External Endpoint pointing to `WorkdayTranslateFunction`, build a Translation Orchestration that passes text + target language and parses the response, optionally wire it to a Business Process (e.g., job description submission or support request creation).

Testing the Deployment:

Local:

```
sam local invoke TranslateFunction --event events/test-event.json
```

Sample event:

```
{
  "text": "Hello world, this is a test translation",
  "sourceLanguage": "en",
  "targetLanguage": "es"
}
```

Expected:

```
{
  "statusCode": 200,
  "body": "{\\"translatedText\\": \\"Hola mundo, esta es una traducci\u00f3n de prueba\\", \\"sourceLanguage\\": \\"en\\", \\"targetLanguage\\": \\"es\\"}"
}
```

Auto-detection test (omit sourceLanguage):

```
{
  "text": "Bonjour le monde, ceci est un test",
  "targetLanguage": "en"
}
```

The function detects French automatically and translates to English.

AWS:

```
aws lambda invoke \
  --function-name WorkdayTranslateFunction \
  --payload file://test/payload.json \
  --region us-west-2 \
  output.json

cat output.json
```

3.3. Demo 3: Amazon Bedrock AgentCore — Employee Onboarding Sample

Blog: https://builder.aws.com/post/3E2Vh8qBCzGyOtZk1gH61PkC10r_p/building-an-ai-employee-onboarding-agent-with-amazon-bedrock-agentcore

GitHub Repo: <https://github.com/aws-samples/sample-workday-aws-integrations/blob/main/workday-extend-examples/docs/blog-03-ai-onboarding-agent.md>

This sample deploys an AI-powered onboarding assistant using Amazon Bedrock AgentCore. It wires up a Strands agent with MCP tools, OAuth2 auth, and streaming responses — all in one make deploy.

What it demonstrates:

- Deploying a containerized agent on AgentCore Runtime
- Tool discovery and invocation via MCP Gateway
- OAuth2 client-credentials authentication (Cognito)
- Lambda-backed MCP tools with mock HR data
- Streaming SSE responses parsed into structured output

How it works: A CLI authenticates via Cognito → sends a prompt to the AgentCore Runtime → Claude Haiku 4.5 calls MCP tools (employee lookup, IT asset check) through the Gateway → Lambda returns data → the agent writes a structured onboarding package streamed back to the client.

Quick Installation: In CloudShell:

1. `git clone https://github.com/aws-samples/sample-workday-aws-integrations.git`
2. `cd sample-workday-aws-integrations/workday-extend-examples/employee-onboarding-agentcore-sample`
3. `make deploy`
4. `make smoke-test`
5. `make cli PROMPT="onboard jane doe as software engineer"`

For detailed instructions, see the README.md (<https://github.com/aws-samples/sample-workday-aws-integrations/tree/main/workday-extend-examples/employee-onboarding-agentcore-sample>)

3.4. Demo 4: AI Gateway to Amazon Bedrock with API Gateway

Blog: <https://aws.amazon.com/blogs/architecture/building-an-ai-gateway-to-amazon-bedrock-with-amazon-api-gateway/>

GitHub Repo: <https://github.com/aws-samples/sample-ai-gateway-for-amazon-bedrock>

This demo deploys an AI gateway using Amazon API Gateway in front of Amazon Bedrock. It enables you to call Bedrock foundation models (e.g., Claude) through a standard REST endpoint.

What it demonstrates:

- API Gateway as a managed "front door" to Amazon Bedrock
- Lambda integration that signs and forwards requests to Bedrock endpoints
- Streaming responses (ConverseStream API) delivered in real-time

AWS Workshop: Storage, Compute & AI Orchestration

- Using boto3 with unsigned requests through a custom endpoint URL
- Enterprise patterns: authorization, throttling, WAF integration (optional)

Deploying with API Gateway as REGIONAL (Hackathon Setup)

⚠ IMPORTANT: The following instructions deploy the API Gateway as a publicly accessible REGIONAL endpoint with authorization disabled. This is for DEMO/HACKATHON PURPOSES ONLY and should NOT be used in production.

The default CloudFormation template deploys the API Gateway as PRIVATE (VPC-only). For hackathon convenience, we will deploy as REGIONAL so you can call the endpoint from anywhere, including from Extend. However, the Cloudformation template has a Rules section that enforces "REGIONAL endpoints must have an authorizer enabled." We will remove that rule to allow REGIONAL + no authorizer.

Deployment Steps:

1. Download the CloudFormation template:

```
curl -fsSL -o bedrock-llm-gateway.yaml https://raw.githubusercontent.com/aws-samples/sample-ai-gateway-for-amazon-bedrock/main/bedrock-llm-gateway.yaml
```

2. Edit the YAML file — Remove the Rules section (Lines 46–51):

Delete the "Rules:" block that contains the "RegionalEndpointRequiresAuthorizer" assertion. This is what enforces that REGIONAL endpoints require an authorizer.

3. Deploy the CloudFormation stack:

```
aws cloudformation create-stack \
  --stack-name bedrock-llm-gateway \
  --template-body file://bedrock-llm-gateway.yaml \
  --parameters ParameterKey=EnableAuthorizer,ParameterValue=false \
  ParameterKey=EndpointType,ParameterValue=REGIONAL \
  --capabilities CAPABILITY_IAM CAPABILITY_NAMED_IAM CAPABILITY_AUTO_EXPAND
```

4. In the CloudFormation console, wait for the stack status to reach CREATE_COMPLETE (3–5 minutes). Then go to CloudFormation → Outputs and copy the GatewayUrl value.

Testing the Deployment:

Follow the "Testing the deployment" section of the blog post, starting from the step to create the `boto3_client_factory.py` file. Skip the "Create VPC environment" Step 2 — it is not needed with a REGIONAL endpoint.

Study the `test_converse_stream.py` file for a Python sample of how you can invoke the Bedrock models through the API Gateway endpoint

Clean up (after Hackathon):

Be sure to clean up after the Hackathon!

```
aws cloudformation wait stack-delete-complete --stack-name bedrock-llm-gateway
```

4. Additional Samples

You can Use CloudFormation (or CLI/Python) to stand up S3, Lambda, API Gateway, and deploy an API Gateway front end and Lambda to access Bedrock models as a REST endpoint. You can work in AWS Cloud9 or from your local terminal (Windows/macOS).

4.1. CloudFormation Fundamentals

What is CloudFormation?

An "infrastructure as code" service where you define resources (buckets, functions, roles) in a template, then deploy as a stack.

- **Template (YAML/JSON):** declares resources
- **Stack:** live deployment of those resources
- **Parameters:** user-supplied inputs (e.g., bucket name)
- **Outputs:** returning values (e.g., ARNs for later use)

 **Tip:** Use `aws cloudformation deploy` to handle parameter prompting, change detection, and IAM capability checks.

4.2. Provision an Amazon S3[®] Bucket

What is S3?

Object storage with 99.999999999% durability. Ideal for files, backups, and static assets.

Why versioning?

Keeps historical copies—helps recover from accidental overwrites/deletions.

⚠ **Important:** S3 bucket names must be globally unique. Add a unique suffix (e.g., your initials + timestamp).

Create s3-bucket.yaml and Deploy S3 bucket

1. Save the template

```
cat > s3-bucket.yaml << 'EOF'
AWSTemplateFormatVersion: '2010-09-09'
Description: S3 bucket in us-west-2 with versioning enabled.

Parameters:
  BucketName:
    Type: String
    Description: Globally unique bucket name (e.g., yourname-uniquebucket-123)

Resources:
  MyBucket:
    Type: AWS::S3::Bucket
    Properties:
      BucketName: !Ref BucketName
      VersioningConfiguration:
        Status: Enabled

Outputs:
  BucketName:
    Description: Name of the created S3 bucket
    Value: !Ref BucketName
EOF
```

2. Deploy (make sure your bucket name below is globally unique)

```
aws cloudformation deploy \
  --template-file s3-bucket.yaml \
  --stack-name s3-demo \
  --parameter-overrides BucketName=yourname-uniquebucket-123
```

3. Verify

```
aws s3api get-bucket-location --bucket yourname-uniquebucket-123
```

You should see the response: us-west-2

Your S3 bucket endpoint will be:

https://yourname-uniquebucket-123.s3.us-west-2.amazonaws.com

Troubleshoot: If you see `BucketAlreadyExists`, choose a new unique name.

4.3. Deploy a "Hello from Lambda" Function

What is Lambda?

Serverless compute: run code on-demand without provisioning servers. You're billed by execution time and memory.

Create `lambda-hello.yaml` and Deploy Lambda

1. Save the template

```
cat > lambda-hello.yaml << 'EOF'
AWSTemplateFormatVersion: '2010-09-09'
Description: Deploys Python 3.9 Lambda that returns "Hello from
Lambda."

Resources:
  LambdaExecutionRole:
    Type: AWS::IAM::Role
    Properties:
      AssumeRolePolicyDocument:
        Version: '2012-10-17'
        Statement:
          - Effect: Allow
            Principal: { Service: lambda.amazonaws.com }
            Action: sts:AssumeRole
      ManagedPolicyArns:
        - arn:aws:iam::aws:policy/service-
          role/AWSLambdaBasicExecutionRole

  HelloFunction:
    Type: AWS::Lambda::Function
    Properties:
      FunctionName: hello-from-lambda
      Runtime: python3.9
      Handler: index.handler
      Role: !GetAtt LambdaExecutionRole.Arn
      Code:
        ZipFile: |
          def handler(event, context):
            return {"statusCode":200,"body":"Hello from Lambda"}
```

```
Outputs:
  FunctionName:
    Description: Name of the deployed Lambda
    Value: !Ref HelloFunction
EOF
```

2. Deploy

```
aws cloudformation deploy \
  --template-file lambda-hello.yaml \
  --stack-name lambda-demo \
  --capabilities CAPABILITY_IAM
```

3. Invoke the Lambda

```
aws lambda invoke \
  --function-name hello-from-lambda \
  --payload '{}' response.json
```

4. Inspect response

```
cat response.json
```

You should get this response:

```
{"statusCode": 200, "body": "Hello from Lambda"}
```

You've successfully deployed a Hello World Lambda function and invoked it.

Your Lambda endpoint will be:

```
https://lambda.us-west-2.amazonaws.com
```

 **Pro tip:** If using Cloud9, in Cloud9's AWS Resources panel, expand Lambda, right-click `hello-from-lambda` → Invoke → view response in the IDE terminal.

4.4. Expose Lambda via AWS API Gateway

What is API Gateway?

A fully managed service that creates, publishes, and secures REST, HTTP, and WebSocket APIs at any scale. Acts as the "front door" for applications to access backend services like Lambda.

Why integrate with Lambda?

- **HTTP Endpoints:** Turn Lambda functions into web APIs accessible via standard HTTP requests
- **Built-in Features:** Authentication, rate limiting, caching, request/response transformation
- **Serverless:** No infrastructure to manage, pay only for API calls

Create api-gateway-lambda.yaml and Deploy API Gateway

1. Save the template

```
cat > api-gateway-lambda.yaml << 'EOF'
AWSTemplateFormatVersion: '2010-09-09'
Description: API Gateway REST API integrated with Lambda function

Resources:
  # API Gateway REST API
  HelloApi:
    Type: AWS::ApiGateway::RestApi
    Properties:
      Name: hello-lambda-api
      Description: REST API for Hello Lambda function
      EndpointConfiguration:
        Types:
          - REGIONAL

  # API Gateway Resource (path)
  HelloResource:
    Type: AWS::ApiGateway::Resource
    Properties:
      RestApiId: !Ref HelloApi
      ParentId: !GetAtt HelloApi.RootResourceId
      PathPart: hello

  # API Gateway Method (GET)
  HelloMethod:
    Type: AWS::ApiGateway::Method
    Properties:
```

```

    RestApiId: !Ref HelloApi
    ResourceId: !Ref HelloResource
    HttpMethod: GET
    AuthorizationType: NONE
    Integration:
      Type: AWS_PROXY
      IntegrationHttpMethod: POST
      Uri: !Sub
        - arn:aws:apigateway:${AWS::Region}:lambda:path/2015-03-
31/functions/${LambdaArn}/invocations
        - LambdaArn: !Sub
arn:aws:lambda:${AWS::Region}:${AWS::AccountId}:function:hello-from-
lambda

# Lambda Permission for API Gateway
LambdaApiPermission:
  Type: AWS::Lambda::Permission
  Properties:
    FunctionName: hello-from-lambda
    Action: lambda:InvokeFunction
    Principal: apigateway.amazonaws.com
    SourceArn: !Sub arn:aws:execute-
api:${AWS::Region}:${AWS::AccountId}:${HelloApi}/*/*

# API Gateway Deployment
ApiDeployment:
  Type: AWS::ApiGateway::Deployment
  DependsOn: HelloMethod
  Properties:
    RestApiId: !Ref HelloApi
    StageName: prod

Outputs:
  ApiEndpoint:
    Description: API Gateway endpoint URL
    Value: !Sub https://${HelloApi}.execute-
api.${AWS::Region}.amazonaws.com/prod/hello

```

```
    ApiId:
      Description: API Gateway ID
      Value: !Ref HelloApi
EOF
```

2. Deploy the stack

```
aws cloudformation deploy \
  --template-file api-gateway-lambda.yaml \
  --stack-name api-gateway-demo \
  --capabilities CAPABILITY_IAM
```

3. Get the API endpoint

```
aws cloudformation describe-stacks \
  --stack-name api-gateway-demo \
  --query 'Stacks[0].Outputs[?OutputKey==`ApiEndpoint`].OutputValue' \
  --output text
```

4. Test the endpoint

```
curl $(aws cloudformation describe-stacks \
  --stack-name api-gateway-demo \
  --query 'Stacks[0].Outputs[?OutputKey==`ApiEndpoint`].OutputValue' \
  --output text)
```

Expected Response:

```
{"statusCode": 200, "body": "Hello from Lambda"}
```

Understanding the Integration

Key Components:

Component	Purpose
<code>AWS::ApiGateway::RestApi</code>	Creates the API container
<code>AWS::ApiGateway::Resource</code>	Defines URL path (<code>/hello</code>)

Component	Purpose
<code>AWS::ApiGateway::Method</code>	Defines HTTP method (GET) and Lambda integration
<code>AWS::Lambda::Permission</code>	Allows API Gateway to invoke Lambda
<code>AWS::ApiGateway::Deployment</code>	Publishes API to a stage (<code>prod</code>)

Integration Flow:

Client Request (GET /hello)

↓

API Gateway (`https://xxxxx.execute-api.us-west-2.amazonaws.com/prod/hello`)

↓

Lambda Function (`hello-from-lambda`)

↓

Response (`{"statusCode": 200, "body": "Hello from Lambda"}`)

KEY INSIGHT: The `AWS_PROXY` integration type passes the entire HTTP request to Lambda and expects Lambda to return a properly formatted HTTP response.

Test in Browser

Your API endpoint is now publicly accessible:

1. Copy the endpoint URL from the deployment output
2. Paste into any web browser
3. You should see: `Hello from Lambda`

For Extend calls, your API Gateway endpoint will be:

`https://[api-id].execute-api.us-west-2.amazonaws.com/prod/hello`

⚠ IMPORTANT: Deploy the Lambda function (Section 3) **before** deploying API Gateway, as the integration references the existing Lambda function.

5. Troubleshooting: Top Tips

1. Bucket name collision

- Pick a unique suffix: `<yourname>-bucket-$(date +%s)`.
- 2. **Missing IAM capability**
 - Always add `--capabilities CAPABILITY_IAM` when stacks create roles.
- 3. **Invalid credentials**
 - **Cloud9:** verify AWS Settings → Credentials are on.
 - **Local:** re-run `aws configure` with valid keys.
- 4. **Script errors**
 - `EntityAlreadyExistsException` is normal on reruns.
 - Ensure `pip3 install boto3` completed successfully.
- 5. **Agent not appearing**
 - Wait 30–60 seconds after creation for console propagation.
- 6. **Trouble installing Python requirements.txt**
 - If you are running a Python version earlier than 3.10 you may get an error when installing dependencies.
 - **Option 1:** Upgrade your Python version to 3.10+ to use ipython 8.29.0
 - **Option 2:** Use a compatible version of ipython for Python 3.9
 - Update `src/requirements.txt`
 - Line #7: Change
from `ipython==8.29.0` to `ipython<3.10`
- 7. **ImportError invoking Bedrock Agents**
 - If you ran into an ImportError when deploying Bedrock Agents, it may be due to you running Python earlier than 3.10. You'll need to modify a file as a workaround as follows:
 - Update `src/utils/bedrock_agent.py`
 - Replace Line #33: `from typing import Self, Callable, Union`
 - with:
 - `from typing import TypeVar, Callable, Union`
`Self = TypeVar('Self', bound='ParameterSchema._Param')`
- 8. **API Gateway 403 Forbidden**
 - Check Lambda permission - API Gateway needs `lambda:InvokeFunction`
- 9. **API Gateway 500 Internal Server Error**
 - Verify Lambda function exists and is named `hello-from-lambda`
- 10. **API Gateway endpoint not found**
 - Wait 30-60 seconds after deployment for propagation

11. CORS errors (browser)

- Add CORS configuration to API Gateway method (not covered in basic setup)

6. Further Reading

- **CloudFormation:** <https://docs.aws.amazon.com/cloudformation>
- **Lambda:** <https://docs.aws.amazon.com/lambda>
- **API Gateway:** <https://docs.aws.amazon.com/lambda/latest/dg/services-apigateway-tutorial.html>
- **Try more Bedrock Samples:** <https://github.com/awslabs/amazon-bedrock-agent-samples/tree/main>

Happy building! 🚀